

Exercise 1: Setting Up JUnit



As shown in the above picture a new java project named “JUnitDemo” is created and the JUnit4 libraries are added to this java project.

Calculator.java:

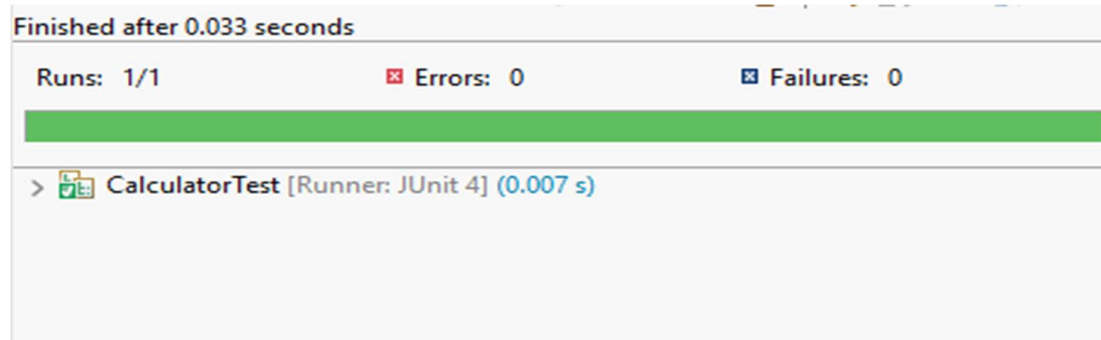
```
public class Calculator {  
  
    public int add(int a, int b) {  
  
        return a + b;  
  
    }  
  
}
```

CalculatorTest.java:

```
import static org.junit.Assert.assertEquals;  
  
import org.junit.Test;  
  
public class CalculatorTest {  
  
    @Test  
  
    public void testAdd() {  
  
        Calculator calc = new Calculator();  
  
        assertEquals(5, calc.add(2, 3));  
  
    }  
  
}
```

```
}
```

Output:



Exercise 3: Assertions in JUnit

```
import static org.junit.Assert.*;
```

```
import org.junit.Test;
```

```
public class AssertionsTest {
```

```
    @Test
```

```
    public void testAssertions() {
```

```
        assertEquals(5, 2 + 3);
```

```
        // Assert Not Equals
```

```
        assertNotEquals(6, 2 + 3);
```

```
        // Assert True
```

```
        assertTrue(5 > 3);
```

```
        // Assert False
```

```
assertFalse(5 < 3);
```

```
// Assert Null
```

```
assertNull(null);
```

```
// Assert Not Null
```

```
assertNotNull(new Object());
```

```
// Assert Same
```

```
String str = "JUnit";
```

```
assertSame(str, str);
```

```
// Assert Not Same
```

```
assertNotSame(new String("Java"), new String("Java"));
```

```
// Assert Array Equals
```

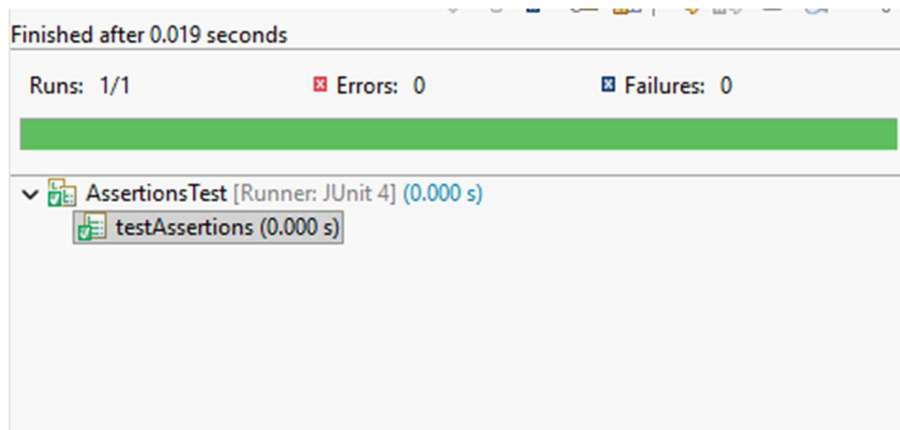
```
int[] expected = {1, 2, 3};
```

```
int[] actual = {1, 2, 3};
```

```
assertArrayEquals(expected, actual);
```

```
}
```

```
}
```



Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

```
import static org.junit.Assert.*;
```

```
import org.junit.After;
```

```
import org.junit.Before;
```

```
import org.junit.Test;
```

```
public class AAATest {
```

```
    private Calculator calculator;
```

```
    @Before
```

```
    public void setUp() {
```

```
        System.out.println("Setup: Creating Calculator object");
```

```
        calculator = new Calculator();
```

```
    }
```

```
    @After
```

```
    public void tearDown() {
```

```
System.out.println("Teardown: Cleaning resources");  
calculator = null;  
}
```

```
@Test
```

```
public void testAddition() {
```

```
    // Arrange
```

```
    int a = 10;
```

```
    int b = 20;
```

```
    // Act
```

```
    int result = calculator.add(a, b);
```

```
    // Assert
```

```
    assertEquals(30, result);
```

```
}
```

```
@Test
```

```
public void testMultiplication() {
```

```
    // Arrange
```

```
    int a = 5;
```

```
    int b = 4;
```

```
    // Act
```

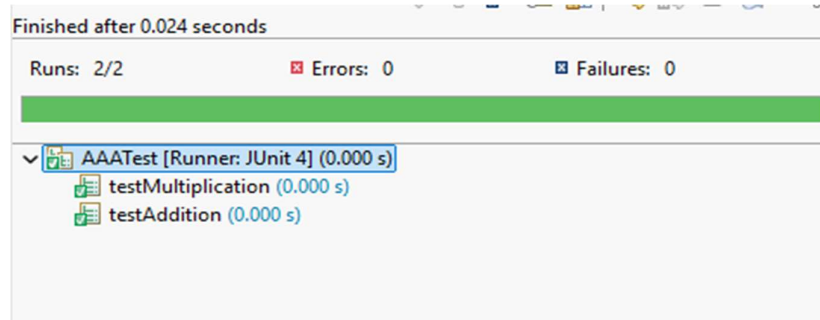
```
    int result = calculator.multiply(a, b);
```

```

    // Assert

    assertEquals(20, result);
}
}

```



Problems @ Javadoc Declaration Console X

```

<terminated> AAATest [JUnit] C:\Users\Asus\.p2\pool\plugins\org.eclipse
Setup: Creating Calculator object
Teardown: Cleaning resources
Setup: Creating Calculator object
Teardown: Cleaning resources

```

Exercise 1: Mocking and Stubbing:

ExternalApi.java

```

public interface ExternalApi {

    String getData();

}

```

MyService.java

```

public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {

        this.api = api;

    }

    public String fetchData() {

        return api.getData();

    }

}

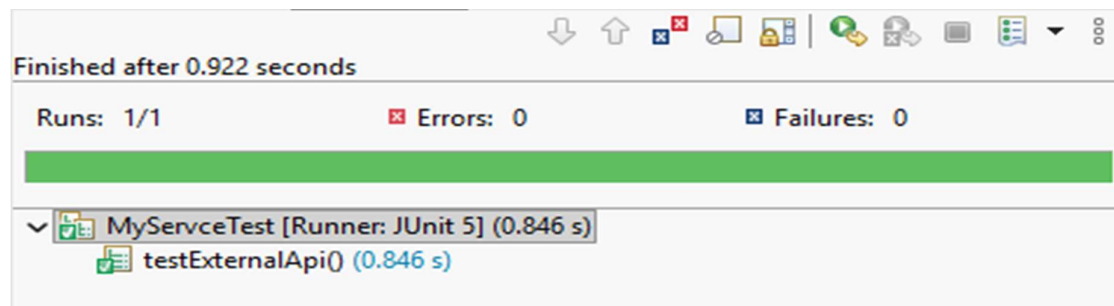
```

```
}  
}
```

MyServiceTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;  
  
import static org.mockito.Mockito.*;  
  
import org.junit.jupiter.api.Test;  
  
public class MyServiceTest {  
  
    @Test  
  
    void testExternalApi() {  
  
        ExternalApi mockApi = mock(ExternalApi.class);  
  
        when(mockApi.getData()).thenReturn("Mock Data");  
  
        MyService service = new MyService(mockApi);  
  
        String result = service.fetchData();  
  
        assertEquals("Mock Data", result);  
  
        verify(mockApi).getData();  
  
    }  
  
}
```

OUTPUT:



Exercise 2: Verifying Interactions:

ExternalApi.java

```
public interface ExternalApi {  
    String getData();  
}
```

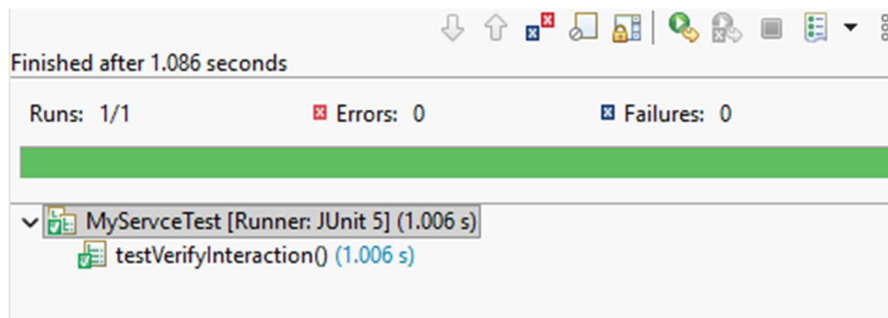
MyService.java

```
public class MyService {  
    private ExternalApi api;  
    public MyService(ExternalApi api) {  
        this.api = api;  
    }  
    public String fetchData() {  
        return api.getData();  
    }  
}
```

MyServiceTest.java

```
import static org.mockito.Mockito.*;  
import org.junit.jupiter.api.Test;  
public class MyServiceTest {  
    @Test  
    public void testVerifyInteraction() {  
        ExternalApi mockApi = mock(ExternalApi.class);  
        MyService service = new MyService(mockApi);  
        service.fetchData();  
        verify(mockApi).getData();  
    }  
}
```


OUTPUT:



Exercise 1: Logging Error Messages and Warning Levels:

LoggingExample.java:

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
public class LoggingExample {
```

```
    private static final Logger logger =
```

```
        LoggerFactory.getLogger(LoggingExample.class);
```

```
    public static void main(String[] args) {
```

```
        logger.error("This is an error message");
```

```
        logger.warn("This is a warning message");
```

```
    }
```

```
}
```

OUTPUT:

Problems @ Javadoc Declaration Console X

<terminated> LoggingExample [Java Application] C:\Users\Asus\p2\pool\plugins\org.eclipse.justj.o

20:26:25.123 [main] ERROR LoggingExample -- This is an error message

20:26:25.125 [main] WARN LoggingExample -- This is a warning message